Dynamic Programming for Minimum Path in a DAG

This solution finds the minimum-cost path from node A (index 0) to node J (index 9) in a Directed Acyclic Graph (DAG) represented by an adjacency matrix. It uses a bottom-up dynamic programming (DP) approach.

1. State Definition

- We maintain an array `states` of length n.
- Each `states[i]` stores:
  - `totalCost`: the minimum cost to reach the sink (J) starting from node i.
  - `to`: the next label on that optimal path from i.
  - `from`: the label of node i (for clarity).

2. Base Case

- The sink node (J at index n−1) has `totalCost = 0` and no outgoing edge.
- Initialized with:

```
Js                                  Copy

states[n - 1] = { from: "", to: "", totalCost: 0 }
```

3. Recurrence Relation

For each node i from right to left:

1. Initialize `totalCost = ∞` and `to = ""`.
2. For every outgoing edge `i → j` (where `data[i][j] > 0`):
   - Compute `newCost = data[i][j] + states[j].totalCost`.
   - If `newCost < states[i].totalCost`, update:

```
Js                                  Copy

states[i].totalCost = newCost
states[i].to = labels[j]
```

3. After scanning all j>i, `states[i]` holds the optimal suffix cost and next hop.

4. Bottom-Up Computation

Js Copy

```
for (let i = n - 2; i >= 0; i--) {
  states[i] = { from: labels[i], to: "", totalCost: Number.MAX_SAFE_INTEGER };

  for (let j = i + 1; j < n; j++) {
    if (data[i][j] === 0) continue;

    const newCost = data[i][j] + states[j].totalCost;
    if (newCost < states[i].totalCost) {
      states[i].totalCost = newCost;
      states[i].to = labels[j];
    }
  }
}
```

- We iterate **backwards** because each subproblem at `i` depends only on solutions for indices `> i`.
- This guarantees all `states[j]` are computed before use.

5. Path Reconstruction

Once `states` is populated, reconstruct the path:

Js Copy

```
const path = [labels[0]];
let current = 0;

while (states[current].to) {
  const nextLabel = states[current].to;
  path.push(nextLabel);
  current = labels.indexOf(nextLabel);
}
```

- Start at A (index 0).
- Follow the `to` pointers until you reach the sink.

Final output:

Copy

```
Minimum Cost:  <states[0].totalCost>
Path: A -> … -> J
```

6. Time and Space Complexity

- Time: $O(n^2)$ since for each of the n nodes we scan up to n successors.
- Space: O(n) for the `states` array and O(1) extra in path reconstruction.

Why DP Works Here

- The DAG structure (no cycles, edges only forward) ensures subproblems don't overlap in a circular way.
- Bottom-up DP leverages already solved suffix costs, avoiding repeated work.
- We only scan each possible edge once, achieving an optimal $O(n^2)$ solution compared to exponential backtracking.